
Security Playground

Project Plan

Nikola Panayotov

Table of Contents

1 Project description.....	0
1.1 Problem and intended users.....	0
2 Scope and constraints.....	0
3 Risks and validation priorities.....	0
4 Project Planning.....	0
4.1 Milestones.....	0
4.2 Design Evolution.....	0

1 Project description

Security Playground is a learning project in which I build a small networked web application, attack it in a safe environment, improve it, and test it again. The application, SecureNotes, lets users create an account, log in, and create, view, edit and delete personal notes. It is built with React and TypeScript on the frontend, Node.js and TypeScript on the backend, and PostgreSQL as the database, and it runs in Docker on Ubuntu Linux. The project produces two versions: an intentionally weak Version 1 that runs only in an isolated environment, and a hardened Version 2 that runs on Ubuntu VPS behind Nginx.

The project answers the challenge question: how can a basic networked web application be built within the Fontys ICT Netlab, attacked, and improved using fundamental Linux or Windows tools? It supports the learning areas Cyber Security Principles & Techniques and Network & Cloud Basics.

1.1 Problem and intended users

The project challenge is to understand, through practice, how web applications and servers become vulnerable and how they are secured. Weak authentication, missing authorization checks, unsafe input handling, insecure network configuration and poor logging are easy to read about but hard to understand without seeing them happen. For me as the developer, the challenge is to build a working application quickly, to test it with fundamental tools such as Nmap, Wireshark, tcpdump and curl, and to document findings, fixes and retest results so that someone else can repeat them.

The intended audience is myself as a learner and the Fontys teachers who assess the evidence. The simulated end users of SecureNotes are normal users whose accounts and notes must be protected from other users and from outsiders. These descriptions come from the project idea and still need to be confirmed with a teacher.

2 Scope and constraints

The first version contains one user role, plain text notes and a single deployment per environment. It covers account registration, login and logout, and the create, view, edit and delete functions for personal notes. Testing covers network exposure (ports, HTTP versus HTTPS traffic), authentication, authorization, SQL injection, cross-site scripting, input validation, logging and server configuration. The plan is to use React, Node.js with Fastify or Express, PostgreSQL, Docker, Nginx and UFW. TypeScript is already familiar to me, so the technology comparison should consider how much time each option leaves for testing and hardening.

Note sharing, file uploads, password reset by email, multi-factor authentication, additional user roles, high availability, custom exploits, malware and denial-of-service tests are outside this version. Optional extras such as Fail2ban or automated test scripts must not delay the core work. Version 1 must never be exposed to the public internet. It runs only in the Fontys ICT Netlab, a local virtual machine or an isolated Docker network, and only systems I own or that are provided for this project are scanned or attacked. Development time is limited by the course schedule, so the Advice document proposes a relative plan that can be aligned with the submission date.

3 Risks and validation priorities

Risk	Effect	Response
Vulnerable version exposed publicly	Real attackers could abuse Version 1 or the VPS.	Run Version 1 only in isolated networks. Check port bindings and scan from outside before every deployment.
Weaknesses that cannot be reproduced	Findings lack evidence and the before and after comparison is weak.	Build a minimal prototype early and confirm each planned weakness. Adjust the design if one cannot be reproduced.

Risk	Effect	Response
Scope growth	Extra features reduce time for testing and retesting.	Complete the Must haves before any optional work. Keep sharing and uploads outside scope.
Too little time for retesting	Version 2 is not compared fairly with Version 1.	Write the test cases before attacking, record evidence while testing, and time-box the build.
Netlab restrictions on tools or scanning	Planned tests cannot run.	Confirm the allowed tools and rules early. Keep a local virtual machine as a fallback.
Unverified generated code	Code may be incorrect, insecure or difficult to explain.	If AI is used, review small changes, check documentation and test each increment.

The validation priorities, in order, are: confirm the Netlab rules and the isolation setup; confirm that each planned weakness can be reproduced in Version 1; run the same test cases against both versions; and verify the exposed ports and HTTPS of Version 2 from outside the VPS.

4 Project Planning

The table below shows the planned progress across the project, structured around the LO2 activities (Analyse, Advise, Design, Realise, Validate). The weeks are relative and can be aligned with the submission date. The status column will be updated as the work progresses, so the table shows what has actually been done and not only what was planned.

LO2 activity	Timing	Status and evidence
Analyse	Week 1, part - 1	Draft written. Problem, scope, requirements and planned weaknesses defined; see Security Playground Analysis. Teacher feedback on scope and Netlab rules still to be confirmed.
Advise	Week 1, part - 2 - Week 2, part -1	Planned. Technology comparison (Fastify versus Express, Docker setup, Nginx and UFW) and the proposed solution; see Security Playground Advice.
Design	Week 2, part - 2	Planned. Architecture for Version 1 and Version 2, database schema, API endpoints and test case design; see Security Playground Design.
Realise	Week 3 - Week 4, part - 1	Planned. Build Version 1 (frontend, backend, database, authentication, notes) and deploy it in the isolated environment. Then build Version 2 with the fixes and deploy it to the VPS.
Validate	Week 4, part - 2	Planned. Test Version 1 with Nmap, Wireshark, tcpdump, curl and browser tools and record the findings. Retest Version 2 with the same cases against the acceptance checks in Security Playground Analysis. Finish with the reflection.

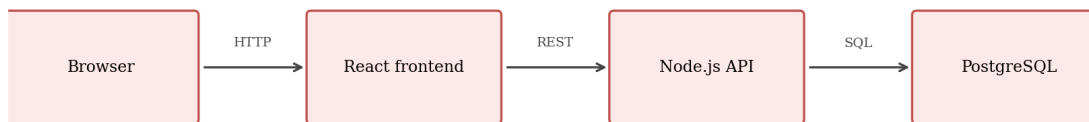
4.1 Milestones

By the end of Week 1, the Analysis is drafted and the Netlab rules and allowed tools are confirmed. By the end of Week 2, Advice and Design are finished, with a final architecture and a set of written test cases. By the end of Week 3, a working Version 1 runs in the isolated environment and every planned weakness has been reproduced. By the end of Week 4, the Version 1 tests are finished and documented, and the Version 2 fixes are implemented and deployed on the VPS. By the end of Week 5, Version 2 has been retested and the before and after results are compared. Week 6 is reserved for the reflection, the final report and any remaining fixes.

4.2 Design Evolution

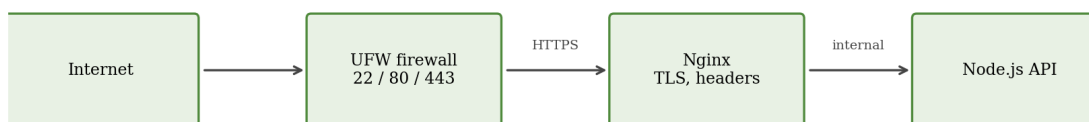
The figure below shows the planned architecture of Version 1 and Version 2. Version 1 is a simple application on an isolated network with plain HTTP and several intentional weaknesses. Version 2 adds the UFW firewall and Nginx with TLS in front of the API, keeps PostgreSQL on the internal Docker network, and applies the fixes for each documented finding. The final design, including the database schema, API endpoints and test cases, will be documented in Security Playground Design.

Version 1: intentionally weak (isolated Netlab / VM / Docker network only)



Plain HTTP, weak passwords, string-built SQL, no rate limiting, extra open ports, minimal logging

Version 2: hardened (Ubuntu VPS)



Also PostgreSQL on the internal Docker network only. Hashing, parameterized queries, ownership checks, rate limiting, security logs